

PIPEANN-FILTER: An Efficient Filtered Vector Search System on SSD

Hao Guo
Tsinghua University
Beijing, China

Jiwu Shu
Tsinghua University
Beijing, China

Youyou Lu*
Tsinghua University
Beijing, China

Abstract

We propose PIPEANN-FILTER, an efficient filtered vector search system on SSD. Unlike existing systems that explore only valid vectors (i.e., those satisfying the attribute constraints) during search, PIPEANN-FILTER explores a *superset* of valid vectors, and performs attribute verification after getting the top- k closest result vectors. This allows PIPEANN-FILTER to leverage probabilistic data structures (e.g., Bloom filters) to identify the superset, trading off a small number of false-positive vector explorations for a massive reduction in SSD I/O for attribute reading. Evaluations show that PIPEANN-FILTER improves search latency and throughput compared to state-of-the-art systems. PIPEANN-FILTER is open-source at <https://github.com/thustorage/PipeANN>.

1 Introduction

Real-world datasets associate high-dimensional vectors with attributes. Vectors capture data semantics [16, 27, 30], while each attribute provides a factual property of variable size. To query such datasets, systems use filtered Approximate Nearest Neighbor Search (*filtered ANNS*), which finds the top- k closest *valid vectors* (i.e., those satisfying attribute constraints) to a query. It is widely used in applications like search [44] and retrieval-augmented generation [12, 21]. For example, an e-commerce platform [23, 44] searches for items that are semantically similar to a reference image, while restricting the results to a specific price range and brand.

Filtered ANNS explores vectors in a graph-based index [5, 15, 25, 29], where vectors represent nodes and are connected by edges. Attribute filtering can occur before ANNS by scanning attributes (*pre-filtering*), or after ANNS by verifying the result vectors (*post-filtering*). The graph-based index also enables *in-filtering*: when exploring a vector, the system filters its neighbors, in order to traverse only the sub-graph of valid vectors. These three mechanisms are complementary; the optimal choice depends on the query selectivity, namely the ratio of valid vectors in the dataset [24].

Traditional ANNS systems store all data in memory. However, as datasets scale to billions of vectors, both academia [6, 37] and industry [41, 44] manage vectors on Solid-State Drives (SSDs) for ANNS. Filtered ANNS datasets share the same scale: Alibaba [44] manages 830 million vectors with 21 attributes each, and LAION400M [31] contains 400 million vectors with 15 attributes each. Therefore, it is crucial to manage both vectors and attributes on SSDs for filtered ANNS; yet, we find this remains underexplored.

Unlike in-memory systems, filtered ANNS on SSD introduces I/O overheads, which make pre- and in-filtering inefficient. For in-filtering, exploring a vector requires reading hundreds of neighbor vectors (for distance comparison) and their attributes (for filtering). While in-memory compressed vectors can mitigate vector reads [37], attribute reads still incur massive random SSD I/O, limiting search throughput. Similarly, pre-filtering requires on-SSD attribute scans, which are highly expensive, especially for multi-attribute constraints. A trivial workaround is to store all attributes in memory. However, this sacrifices the memory efficiency of on-SSD ANNS. For instance, attributes in LAION400M take 60GB, far exceeding the 24GB used by compressed vectors. This issue is more severe for datasets with richer attributes, requiring ~ 300 GB for 500 million e-commerce items [18].

We find that the root cause of this I/O bottleneck is *strict filtering*: Both pre- and in-filtering require every vector explored during the search *must be* valid. Therefore, before exploring any vector, the system must fetch its exact attributes from the SSD, causing the I/O bottleneck.

We argue that strict filtering is unnecessary and propose *speculative filtering*. Instead of filtering and exploring only valid vectors, speculative filtering allows exploring a *superset* of them (allowing false positives), and performs attribute verification after getting the closest result vectors.

Speculative filtering enables a tradeoff: accepting false-positive explorations to reduce attribute reads. We demonstrate how this tradeoff boosts performance through two examples. For in-filtering, speculative filtering can filter neighbors using memory-efficient probabilistic data structures (e.g., Bloom filters for label membership checks), instead of reading their attributes from the SSD. For pre-filtering, instead of scanning all involved attributes, speculative filtering can prune constraints to generate the superset (e.g., executing only the most lightweight branch in a multi-attribute AND constraint). Attribute verification is then executed only for the result vectors of ANNS.

One might assume that reading exact attributes for verification introduces additional I/O overhead. However, we find that it incurs little to no extra I/O: On-SSD ANNS first estimates distances using in-memory compressed vectors, and then fetches the closest full-precision vectors from the SSD for re-ranking. These re-ranked vectors are exactly what speculative filtering verifies. By co-locating a vector and its attributes, a single I/O retrieves both. If both share the same SSD page, reading attributes introduces no extra I/O.

Interestingly, we find that some false-positive explorations improve search accuracy. Since the base ANNS graph is

*Youyou Lu is the corresponding author (luyouyou@tsinghua.edu.cn).

sparse (~ 100 neighbors per node), low query selectivity often leaves nodes without valid neighbors, fragmenting the valid vectors into disconnected sub-graphs. This traps traditional in-filtering, which explores only valid vectors, in local optima, resulting in low accuracy. By exploring invalid vectors, speculative filtering bridges these disconnected sub-graphs, allowing the search to find more valid vectors.

Despite its potential, applying speculative filtering to an on-SSD filtered ANNS system is challenging. First, cost estimation is required to route each query to its suitable filtering mechanism. This is complex because the model must consider both compute and I/O costs, while accounting for false positives. Second, the data structures for getting the superset must simultaneously achieve low SSD I/O, a low false-positive rate, and a small memory footprint.

To tackle these challenges, we design PIPEANN-FILTER, an efficient filtered ANNS system on SSD. For cost estimation, PIPEANN-FILTER computes the "equivalent" search parameters required to find a given number of valid vectors, based on the estimated false-positive rate and query selectivity. Both compute and I/O costs can be estimated using the search parameters [40]. For efficient superset generation, PIPEANN-FILTER achieves memory efficiency and low SSD I/O via a two-level data structure design. It combines in-memory Bloom filters with on-SSD inverted indexes for label constraints [15], and combines in-memory quantized values with on-SSD sorted indexes for range constraints [50]. Furthermore, PIPEANN-FILTER accelerates Boolean combinations (AND, OR) by pruning heavy constraints, and provides a flexible interface to support custom, user-defined attribute constraints.

We evaluate PIPEANN-FILTER to show its efficacy. On million-scale datasets targeting a 0.9 recall, PIPEANN-FILTER reduces latency by at least 89.5% and delivers at least $32.3\times$ higher throughput compared to Milvus [41], a state-of-the-art vector database using pre-filtering. Compared to PipeANN-BaseFilter [17], an optimized baseline that dynamically routes queries to pre- or post-filtering, PIPEANN-FILTER achieves at least $1.71\times$ higher throughput with comparable latency. This performance boost scales to a 100-million-vector dataset. Furthermore, against Filtered-DiskANN [15], a strict in-filtering system using customized graph structures, PIPEANN-FILTER achieves $4.35\times$ higher throughput and reduces latency by 77.6%. Notably, PIPEANN-FILTER also achieves higher peak recall, as its false-positive explorations improve graph connectivity.

In summary, this paper makes the following contributions:

- We analyze the I/O overheads of filtered ANNS on SSD, identifying strict filtering as the root cause (§2).
- We argue that strict filtering is unnecessary and propose speculative filtering, which allows exploring a superset of valid vectors. This enables a tradeoff between false-positive vector exploration and attribute-filtering I/O (§3).

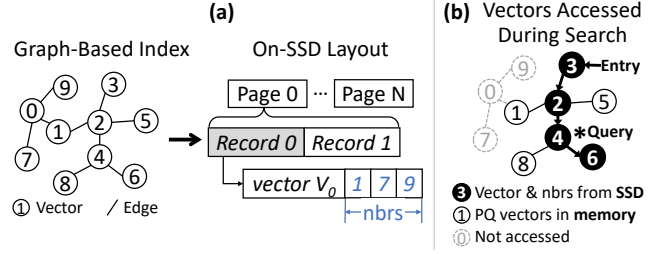


Figure 1. Overview of an on-SSD graph-based ANNS index. (a) On-SSD data layout. Each record stores a full-precision vector and its neighbor IDs. (b) Vector access pattern during a search. Records of vectors along the search path are fetched from the SSD. Their neighbors’ PQ-compressed vectors are accessed in memory for distance comparison, without involving the SSD. Other vectors are not accessed.

- We design PIPEANN-FILTER, a filtered ANNS system on SSD. Leveraging speculative filtering, PIPEANN-FILTER supports arbitrary attribute constraints while reducing SSD I/O via memory-efficient probabilistic data structures (§4).
- We evaluate PIPEANN-FILTER to demonstrate its state-of-the-art efficacy in on-SSD filtered ANNS (§5).

2 Background and Motivation

In this section, we first introduce on-SSD graph-based ANNS. We then describe filtered ANNS mechanisms and analyze their overheads when deployed on the SSD.

2.1 On-SSD Graph-based ANNS

Given a query vector, Approximate Nearest Neighbor Search (ANNS) finds its top- k nearest *base vectors* (vectors in the dataset). The search is approximate for efficiency, meaning that it may only find a subset of the exact top- k vectors.

Index structure. As shown in Figure 1(a), a graph-based ANNS index organizes base vectors into a directed graph, where each vector represents a node and is connected by edges. On the SSD, this graph is stored as a set of adjacency lists. We define each adjacency list as a *record*, which contains a full-precision vector and its out-neighbor IDs (e.g., V_0 and its neighbors 1, 7, 9). In memory, the index maintains compressed vectors using product quantization (PQ) [14] to reduce SSD reads during a search. Typically, the memory-to-SSD space ratio is 1:10 [37].

Search algorithm. Graph-based ANNS follows a best-first search algorithm [13, 26, 37]. As shown in Figure 1(b), starting from an entry node (Node 3), the search navigates toward the query vector step by step. In each step, the algorithm explores one vector in the *candidate pool*: it fetches the current closest vector and its neighbor IDs from the SSD (in a

single record). Using the neighbor IDs, it retrieves the PQ-compressed neighbor vectors from memory and computes their approximate distances to the query, without involving the SSD. Then, it adds the neighbors to the candidate pool, sorts them using PQ distance, and decides which vector to explore in the next step. The search terminates upon reaching a local optimum, specifically when the top- L vectors in the candidate pool can no longer be updated with closer neighbors. The explored vectors are re-ranked using their full-precision distances to produce the results.

2.2 Filtered ANNS

Given a query vector, filtered ANNS finds the top- k nearest base vectors that satisfy specific *attribute constraints* (we call these *valid* vectors). In this scenario, a base vector is associated with several *attributes* of various data types, such as integers, strings, or lists of labels. An attribute constraint defines a condition an attribute must meet (e.g., falling within an $[l, r]$ range, matching a string prefix, or containing a specific label). Multiple attribute constraints are connected using Boolean expressions (e.g., AND/OR).

The `is_member` abstraction. In this paper, we focus on filtered ANNS with *arbitrary attribute constraints*. We abstract these constraints into a single Boolean function: it takes the query’s and a base vector’s attributes as input, and outputs true if the vector is valid, or false otherwise. Following existing systems [9, 29], we call this function `is_member`.

Filtered ANNS mechanisms. There are three mechanisms for filtered ANNS: post-filtering, pre-filtering, and in-filtering. Their efficiency varies with different *query selectivity* (i.e., the ratio of valid vectors in the dataset) [24, 25].

- *Post-filtering* first performs ANNS to get an initial vector set with more than k vectors, and then filters the set to keep only the valid ones. It is suitable for high query selectivity, where valid vectors are abundant.
- *Pre-filtering* first scans the attributes to collect the valid vector set, and then performs a brute-force exact NNS on this set. It is suitable for low query selectivity, where post-filtering would require an exhaustively long graph traversal to find enough valid vectors.
- *In-filtering* applies the filter dynamically during the ANNS process. During graph traversal, it only explores valid vectors. It is suitable for moderate query selectivity: in this case, post-filtering still has high traversal overhead, while pre-filtering’s brute-force NNS becomes expensive on the moderately sized subset.

2.3 Analyzing Filtered ANNS on SSD

In real-world vector datasets, vectors are usually associated with multiple attributes, which consume significant space. For example, in the LAION400M dataset [31], each vector has 15 attributes (e.g., image description, image width), making up 58.6 GB for 400 million vectors. In Amazon’s e-commerce

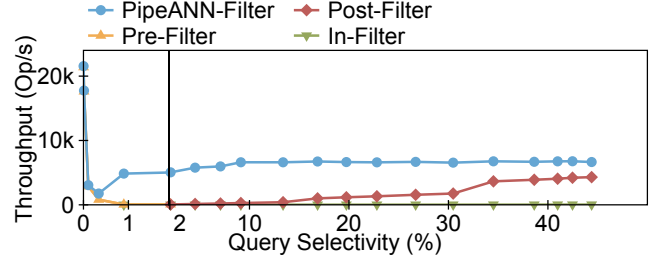


Figure 2. Search throughput of different filtering mechanisms across varying selectivities. Dataset: LAION100M. Target recall: 0.9. The blue line shows the performance of our system, PIPEANN-FILTER.

dataset [18], 500 million items contain ~300GB attributes (e.g., product features and reviews).

Therefore, to support large-scale vectors with attributes, it is crucial to *store both vectors and their attributes on SSDs* for filtered ANNS. Unlike existing systems that store attributes in memory [15, 25], checking whether a vector is valid requires reading its attributes from SSD, which incurs significantly higher I/O overheads.

In this paper, we explore **how to achieve efficient filtered ANNS on SSDs**. To understand the I/O overheads of existing filtered ANNS mechanisms, we evaluate them on a 100M-scale dataset, LAION100M, using the range-filtering workload (detailed in §5.1). Figure 2 shows the results:

Post-filtering works, but only for high selectivity. Post-filtering applies the filter to the initial ANNS results, which have already been fetched from the SSD during the search process. By storing the attributes inside the same record as the vector, post-filtering introduces no separate I/O for attributes and thus remains highly efficient. Consequently, Figure 2 shows that it achieves reasonable throughput at high selectivities. However, when the selectivity drops below 10%, its throughput drops to <300 QPS, as it should traverse a substantial part of the graph to find enough valid results.

Pre-filtering is inefficient due to scan and compute. Pre-filtering requires an attribute scan across the dataset to collect valid vectors, followed by an exact NNS step. As shown in Figure 2, pre-filtering only performs well at extremely low selectivities (<1%). Once the selectivity reaches $\geq 1\%$ (requiring an attribute scan and NNS over ≥ 1 million vectors), its throughput degrades to ≤ 100 QPS. This is because, when the selectivity is not extremely low, the exact NNS becomes a CPU bottleneck. While this process can be I/O-efficient by comparing distances using PQ-compressed vectors in memory (and only fetching the top- L vectors from the SSD for re-ranking), computing these distances for millions of compressed vectors remains compute-intensive.

Even when the selectivity is extremely low, the attribute scan may still be a bottleneck. If the query involves multiple attributes or cannot be accelerated by attribute indexes, the

system is forced to perform expensive attribute scans on the SSD just to identify the valid vectors.

In-filtering is inefficient due to random SSD I/O. Recall the access pattern in Figure 1(b): after fetching Record 2, the in-filtering mechanism needs to check if its neighbors (1, 4, 5) are valid, which requires reading their attributes from the SSD. Because each vector typically has 32–128 neighbors that are randomly scattered across the graph [43], reading their attributes translates to massive random SSD reads, which severely degrades search throughput. Consistent with this analysis, Figure 2 shows that in-filtering’s throughput remains consistently <50 QPS, regardless of query selectivity.

Root cause of I/O overheads: Strict filtering. In conclusion, to support arbitrary query selectivity, we must rethink the filtering mechanisms to eliminate these massive I/O overheads. We find that these bottlenecks are fundamental to how current mechanisms operate. Specifically, both pre- and in-filtering enforce *strict filtering*: they guarantee that every vector explored during the search *must be* a valid vector. To maintain this strict guarantee, the algorithm must fetch and check the actual attributes from the SSD for every encountered vector, inevitably causing the I/O bottleneck.

3 Speculative Filtering

We argue that strict filtering is not necessary and propose *speculative filtering*. Instead of exploring only the valid vectors, speculative filtering explores a *superset* of them during pre- and in-filtering, and performs exact verification of attributes after getting the closest result vectors.

Benefits. This design enables a new I/O tradeoff: we trade a slight increase in false-positive explorations for a massive reduction in attribute filtering I/O. We find three key opportunities where this tradeoff boosts search efficiency:

First, to get this superset, we can significantly reduce SSD I/O by employing memory-efficient probabilistic data structures. For example, a Bloom filter can quickly check if a base vector contains the queried tags. This allows us to identify the superset entirely in memory, introducing only a low false-positive rate while bypassing the SSD.

Second, the exact verification can be piggybacked on the final re-ranking phase, often incurring little to no extra I/O. Recall that on-SSD ANNS already fetches full-precision vectors for re-ranking. We pack the attributes into the same record as the vector, and fetch them together during reranking. This is effective because records are typically not page-aligned; their final page often contains significant unused space. When the attributes are placed in this leftover space, fetching them does not incur extra I/O.

Third, during in-filtering, some false-positive explorations are actually useful; they can be repurposed to improve graph connectivity. When query selectivity is moderately low, strict in-filtering prunes many edges, which breaks the connectivity and thus causes early search termination. In speculative

filtering, the false positives naturally act as "bridge" nodes. Even though these nodes are invalid, exploring their neighbors helps reconnect the graph, thus reducing the chance for the search algorithm to fall into local optima.

The `is_member_approx` abstraction. Speculative filtering uses a new function, `is_member_approx`, to get the superset. Unlike the original `is_member` function, which inputs the base vector’s attributes on SSD, `is_member_approx` inputs only the query’s attributes and the base vector’s ID. It guarantees no false negatives: if it returns false, the vector is definitely invalid. If it returns true, the vector *might* be valid (it belongs to the superset). The original `is_member` function is used in the final step for exact verification.

Speculative filtering for pre-filtering and in-filtering. Leveraging speculative filtering, we modify the execution flow of pre-filtering and in-filtering to reduce SSD reads.

Pre-filtering. As shown in Figure 3(a), strict pre-filtering scans all attribute indexes on the SSD to evaluate every constraint (e.g., $f(A)$ and $g(B)$). Instead, speculative filtering gets a superset by evaluating only a partial set of constraints (e.g., $f(A)$). This reduces SSD reads by completely bypassing the index for B . Next, it performs a brute-force NNS using PQ distances in memory to find the top- $(L + \delta)$ vectors. Finally, it fetches these vectors from the SSD for reranking and uses `is_member` for exact verification. Fetching the δ extra vectors incurs additional SSD reads.

In-filtering. Figure 3(b) illustrates how speculative filtering operates during the graph search. In a search step, strict in-filtering reads the attributes of all neighboring nodes (1, 4, and 5) from the SSD. Instead, speculative filtering checks memory-efficient probabilistic data structures to filter out invalid neighbors. This eliminates the SSD reads for neighbor attributes. Since the check is probabilistic, the search might explore and verify some invalid nodes (like node 4), incurring extra reads from the SSD. However, some invalid nodes act as valuable "bridge" nodes. For instance, exploring node 4 enables the search algorithm to reach the valid node 6. This reduces the chance of early search termination and discovers vectors that strict filtering would otherwise miss.

Unifying existing methods. Interestingly, speculative filtering unifies existing filtering methods, which can be viewed as two special cases:

- **Strict pre- and in-filtering** are the strict extreme. Their `is_member_approx` is identical to `is_member` with a zero false-positive rate. This pushes all the filtering work into the graph traversal step and needs no verification.
- **Post-filtering** is the loose extreme. Its `is_member_approx` is a dummy function that always returns true with a maximum $(1 - \text{selectivity})$ false-positive rate. This pushes all the filtering work to the final verification.

Speculative filtering enables new trade-offs between these two extremes.

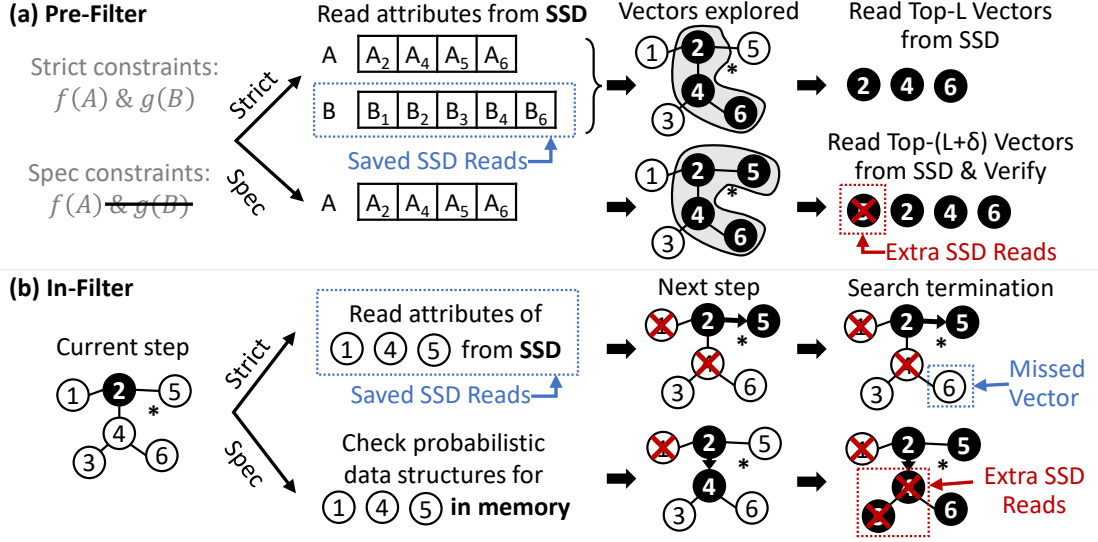


Figure 3. Comparison of speculative pre-/in-filtering with strict pre-/in-filtering.

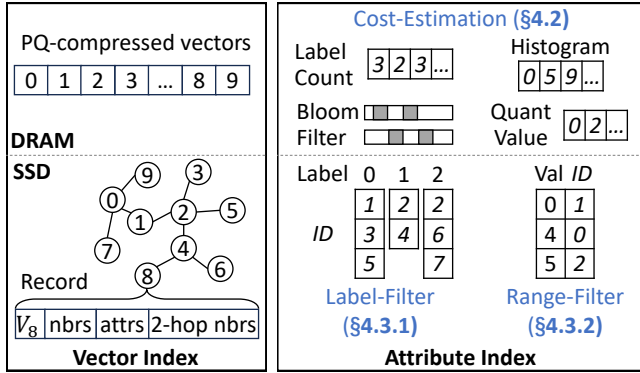


Figure 4. PIPEANN-FILTER overview.

4 PIPEANN-FILTER Design and Implementation

We design and implement PIPEANN-FILTER, a filtered ANNS system on SSD. To build a system atop speculative filtering, PIPEANN-FILTER tackles two main design challenges:

C1: False-positive-aware cost estimation. In-memory filtered ANNS systems [25] directly use query selectivity to estimate query costs and choose filtering mechanisms (e.g., pre-filtering for low selectivity). However, in the speculative filtering scenario, this simple approach is insufficient: First, it should consider the extra search costs caused by false positives. Second, it should consider both SSD I/O and compute overheads simultaneously, which is more complex than in-memory scenarios.

C2: Efficient data structures for attributes. To make `is_member_approx` work, the probabilistic data structures of attributes must meet three requirements: low SSD I/O, a

low false-positive rate, and a small memory footprint. Designing such data structures to support common attribute constraints (e.g., labels and ranges) is under-exploited.

4.1 Overview

Figure 4 shows the overview of PIPEANN-FILTER.

Index layout. PIPEANN-FILTER consists of two parts: the vector index and the attribute index.

The vector index generally follows the graph layout described in §2.1, which stores PQ-compressed vectors in memory and the main graph on the SSD. However, it differs in two key aspects: First, besides the full vector and its neighbors, each record also stores the vector’s attributes, which are used for verification and post-filtering. Second, inspired by ACORN [29], each record also stores a subset of the vector’s 2-hop neighbors (neighbors of its neighbors). Specifically, we randomly select a subset of 2-hop neighbors, making their count 10–20× the number of direct neighbors (e.g., 96 direct neighbors and ~1000 2-hop neighbors). This design improves graph connectivity during speculative in-filtering. PIPEANN-FILTER only reads these extra 2-hop neighbors during in-filtering, and ignores them during pre-filtering and post-filtering.

An attribute index is built only if the attribute’s filtering can be accelerated by an index. The main index structures (e.g., inverted indexes for labels) are stored on the SSD and are scanned during pre-filtering. In memory, PIPEANN-FILTER maintains two types of data structures. First, it uses memory-efficient probabilistic structures (e.g., Bloom filters and quantized values) to support the fast `is_member_approx` function. Second, it keeps statistical summaries (e.g., histograms and label counts) for query cost estimation. Currently, PIPEANN-FILTER provides efficient index designs for

common attribute constraints (such as labels and ranges). However, it can be easily extended to support new, user-defined constraints.

As a result, an indexed attribute is duplicated: one copy is stored column-wise in the attribute index for scan during pre-filtering, while the other is stored row-wise in the vector index records for verification.

Interfaces. A query in PIPEANN-FILTER takes standard vector search parameters (e.g., the query vector and k), plus two new inputs: the query’s attributes and a Selector. PIPEANN-FILTER supports multiple attributes per vector, organized as a key-value (KV) map (e.g., key θ mapped to a range $[l, r)$).

Users define filtering rules using Selector objects, which implement `is_member` and `is_member_approx`. To support query cost estimation, the Selector also exposes functions to estimate query selectivity and false-positive rate.

To optimize performance, the Selector also provides a batched interface called `pre_filter_approx`. Depending on the implementation, this function can either evaluate base vectors one by one using `is_member_approx`, or directly perform a batched scan over an SSD attribute index to quickly return a superset of valid vector IDs. PIPEANN-FILTER leverages this interface in two ways: First, for speculative pre-filtering, this function is used to get the entire valid vector superset. Second, for speculative in-filtering, PIPEANN-FILTER calls this interface to perform a partial attribute scan. For example, by pre-scanning the index for rare labels, it can filter out some invalid vectors before graph traversal. This increases the accuracy of `is_member_approx` during search.

Multiple Selectors can be combined using Boolean logic. For example, an `AndSelector` takes several Selectors and applies the AND operation to their results. Users can also fuse multiple Selectors into a single Selector for optimization. PIPEANN-FILTER provides built-in Selectors for common constraints (e.g., label and range filtering) and standard logical operators (AND/OR) (§4.3). Users can easily extend this interface to support custom constraint types.

Query processing. For each query, PIPEANN-FILTER first executes cost estimation (§4.2) to select the optimal execution strategy: speculative pre-filtering, speculative in-filtering, or post-filtering. This decision balances SSD I/O overhead against memory compute costs. Next, PIPEANN-FILTER executes the search using the chosen method (detailed in §3). If pre- or post-filtering is chosen, PIPEANN-FILTER traverses the standard graph, safely ignoring the 2-hop neighbors.

If speculative in-filtering is chosen, PIPEANN-FILTER leverages 2-hop neighbors to maintain graph connectivity. Specifically, at each search step, the algorithm scans both the direct and 2-hop neighbors to collect up to R nodes that satisfy `is_member_approx` (i.e., possibly valid neighbors). If it finds fewer than R such nodes, it fills the remaining slots with invalid direct neighbors. During graph traversal, the algorithm prefers to explore possibly valid vectors first, even if some

Mechanism	Est. I/O (Pages)	Est. Compute
Pre-filtering	$X_{pre} + \frac{L}{s} \times S_r$	$\frac{sN}{p_{pre}}$
In-filtering (Low s)	$X_{in} + \frac{L}{s} \frac{R}{R_d} \times S_d$	$(\frac{L}{s} \frac{R}{R_d} + \gamma \frac{L}{s}) \times R$
In-filtering (High s)	$X_{in} + \frac{L}{p_{in}} \times S_d$	$\frac{L}{p_{in}} \times (R + \gamma R_d)$
Post-filtering	$\frac{L}{s} \times S_r$	$\frac{L}{s} \times R$

Table 1. Cost estimation for different filtering mechanisms. N : total base vectors. s : estimated query selectivity. p_{pre}, p_{in} : precision of pre/in-filtering (false-positive rate is $1 - p$). X_{pre}, X_{in} : pages read for initial batched attribute index scans (pre_filter_approx, §4.3). R, R_d : out-degree in the standard graph and the graph with 2-hop neighbors. S_r, S_d : record size in the standard graph and the graph with 2-hop neighbors. γ : the relative compute cost of `is_member_approx` compared to a distance computation (we set it to 0.05 by default).

invalid vectors are geographically closer to the query. The search terminates when the top- L valid vectors are exactly verified, and no closer valid neighbors can be found.

4.2 Cost Estimation

Before executing a query, PIPEANN-FILTER uses an analytical cost model to select the optimal filtering mechanism. The decision relies on estimating the expected SSD I/O (pages read) and computation (distance comparisons) for each strategy. The main challenge in this estimation is quantifying the impact of false positives introduced by speculative filtering.

Key principles. To estimate the search overhead, we model the required size of the candidate pool to yield L final valid vectors. Assuming valid vectors are uniformly distributed in the dataset [24], we apply two scaling principles:

1. *Selectivity scaling*: If the query selectivity is s , the search must explore L/s vectors to find L valid ones.
2. *Precision scaling*: If `is_member_approx` has a false-positive rate of $(1 - p)$, namely, the precision is p , false positives take up slots in the candidate pool. Thus, the required pool size scales to L/p .

Importantly, as discussed in §3, false positives are not always pure overhead. During speculative in-filtering with low selectivity, these false positives act as "bridge" edges that maintain graph connectivity. Because the algorithm must traverse these bridge nodes anyway to avoid early termination, the overhead of these specific false positives can be ignored in the cost model.

Formulating the costs. Table 1 summarizes the estimated I/O and compute costs for the different mechanisms.

For *speculative in-filtering*, the behavior has two cases depending on the selectivity. On average, each vector in the densified graph contains sR_d/p_{in} neighbors that satisfy `is_member_approx`. When $sR_d/p_{in} \leq R$ (i.e., low selectivity),

the false positives serve entirely as bridge edges without introducing extra traversal overhead. In this case, the process is equivalent to a standard graph traversal with an effective candidate pool length of $L/s \times R/R_d$. Conversely, when $sR_d/p_{in} > R$ (i.e., high selectivity), the false positives introduce actual overhead. The pool size must be scaled up to L/p_{in} to guarantee L truly valid results.

For *pre-filtering*, the compute cost is dominated by the sN/p_{pre} distance comparisons on PQ-compressed vectors in memory (the re-ranking compute cost is negligible). For *post-filtering*, no index scan is required, and the costs strictly follow the selectivity scaling principle.

PIPEANN-FILTER calculates the estimated cost using a weighted linear model: $\text{Total Cost} = \alpha \times \text{Est. I/O} + \beta \times \text{Est. Compute}$, where α and β are configurable weights. By default, we set $\alpha = 10$ and $\beta = 1$ to reflect the high penalty of SSD accesses.

4.3 Speculative Filtering for Attribute Constraints

In this section, we detail the design of Selectors for common attribute constraints. To fully support PIPEANN-FILTER’s cost estimation and search process, each Selector must provide: (1) `is_member_approx` and batched `pre_filter_approx` interfaces for superset filtering. (2) Functions to estimate the filter’s precision p (or false-positive rate $1 - p$) and query selectivity s (as required in §4.2).

4.3.1 Label Filtering. In label filtering, each vector is associated with a set of categorical labels. A query constraint is typically a Boolean expression of labels (e.g., `label1 OR label2`) [15]. To execute these, we provide `LabelOrSelector` and `LabelAndSelector`. A `LabelOrSelector` checks if a vector contains *at least one* label from the queried labels, while a `LabelAndSelector` checks if it contains *all* of them. For complex Boolean queries, they can be further composed using general `AndSelectors` and `OrSelectors`.

Index structure. On the SSD, labels are stored as inverted indexes: for each label, the IDs of vectors containing it are stored contiguously in ascending order. In memory, we store each label’s offset and total count. This supports fast SSD lookups and selectivity estimation with a minimal memory footprint, as the number of unique labels is typically much smaller than the dataset size. Additionally, we maintain a lightweight, in-memory Bloom filter for each vector to enable fast, probabilistic membership checks.

Implementation of `is_member_approx`. There are two baseline methods for approximate label filtering. The first is to read the inverted indexes of all queried labels from the SSD and merge their vector IDs. This is accurate but causes severe I/O overhead for high-selectivity (frequent) labels. The second is to rely entirely on the in-memory Bloom filters. This requires zero I/O but can become inaccurate due to hash collisions, especially for vectors with many labels.

To balance accuracy and I/O, we take a hybrid approach. Before the graph traversal of speculative in-filtering, we only fetch the inverted indexes of *low-selectivity* (rare) labels from the SSD. We merge these fetched IDs in memory into a single list (using an intersection for `LabelAndSelector`, or a union for `LabelOrSelector`). For the remaining *high-selectivity* (frequent) labels, we fall back to the Bloom filters.

Thus, when `is_member_approx` evaluates a base vector ID during the search, it first performs a binary search to check if the ID exists in this pre-merged target list. If the ID is not in the merged rare-label list, we query the Bloom Filter for `LabelOrSelector`, or return false for `LabelAndSelector`. This hybrid design reduces Bloom filter collisions while keeping SSD I/O strictly bounded.

For speculative pre-filtering, since it is only chosen for queries with overall low selectivity, checking the Bloom filter for every vector in the dataset would be slower than simply scanning the inverted indexes. Thus, its batched `pre_filter_approx` directly reads the required inverted indexes from the SSD and merges the IDs to generate the superset. For `LabelAndSelector`, we further accelerate this process by skipping the scans for high-selectivity (frequent) labels entirely. Because intersecting only the low-selectivity (rare) labels already guarantees a sufficiently small superset, the high-selectivity constraints can be deferred to the final exact verification phase.

Selectivity and precision estimation. We estimate query selectivity using the stored label counts. Assuming label independence, both the intersection selectivity of an AND query and the union selectivity of an OR query are easily computed. To estimate precision, we first estimate the number of true positives using this selectivity. Next, we estimate the number of false positives based on the Bloom filter’s mathematical false-positive rate. The final estimated precision p is the ratio of true positives to the total expected positives.

4.3.2 Range Filtering. In range filtering, the attribute for each dataset vector is a continuous value (e.g., an integer), and the query specifies a target range $[l, r)$.

Index structure. On the SSD, we store the attributes as a flat array of `<vector_ID, value>` pairs, sorted by value. This physical layout allows us to efficiently execute range queries by scanning a contiguous chunk of the array, maximizing sequential SSD reads. (Other structures, like B⁺-trees, could also be easily integrated).

In memory, we maintain two distinct data structures with different granularities for different purposes. For fast per-vector `is_member_approx` checks, we compress the value into a compact, 1-byte integer (256 buckets) for each vector, alongside storing the 256 global bucket boundaries.

For cost estimation, we maintain a separate, more fine-grained summary of the global value distribution. Specifically, this summary is a compact array of 1000 values representing the approximate quantiles of the dataset (e.g., the

0.1-th, ..., 99.9-th percentile values). The key reason a 1000-quantile summary is sufficient for estimation is that our cost model does not require highly sensitive selectivity values for range queries; pre-filtering is chosen as the optimal strategy across the entire low-selectivity case.

Implementation of `is_member_approx`. For speculative in-filtering, `is_member_approx` simply checks the target vector’s 1-byte bucket ID in memory. If the bucket’s value range overlaps with the query range $[l, r)$, it returns true.

For speculative pre-filtering, the batched interface queries the exact SSD index: it uses the in-memory bucket boundaries to quickly locate the starting offset, and then performs a sequential SSD read to fetch the exact valid vector IDs.

Selectivity and precision estimation. We use the 1000-quantile summary to directly estimate the query selectivity. To estimate precision, we use the same summary to estimate the number of true positives. For the total number of positives (which includes false positives), we use the coarse-grained 256-bucket boundaries. Specifically, we count how many coarse buckets overlap with the query range. The estimated precision is the ratio of the two estimates.

4.3.3 Boolean Combination. Complex queries are handled by combining simple Selectors. An `AndSelector` requires the vector to satisfy *all* child constraints, while an `OrSelector` requires it to satisfy *at least one*.

Implementation of `is_member_approx`. For in-filtering, we evaluate all child branches by default, as the in-memory `is_member_approx` checks are highly optimized.

For pre-filtering, we optimize `AndSelectors` using early termination: If a complex query contains a low-selectivity condition AND a high-selectivity condition across different attribute types, evaluating both on the SSD is wasteful. Instead, the `AndSelector` skips the SSD scan for the high-selectivity branch. It only scans the low-selectivity index to generate a safe superset, deferring the high-selectivity condition to the final exact verification phase.

Selectivity and precision estimation. Assuming that constraints are independent [32]: For an `AndSelector`, the combined selectivity is the product of its children’s selectivities. For an `OrSelector`, it is the union of its children’s selectivities. Precision estimation follows a similar logic. The combined precision of an `AndSelector` is the product of the individual precisions. For an `OrSelector`, the combined precision is the union selectivity of the true positives divided by the union selectivity of all returned positives.

5 Evaluation

We evaluate PIPEANN-FILTER to answer the following questions:

- How does PIPEANN-FILTER perform against state-of-the-art systems on million-scale datasets with label filtering? (§5.2)

- How does PIPEANN-FILTER scale to 100-million-scale datasets when handling complex attribute constraints? (§5.3)
- How effective are PIPEANN-FILTER’s core techniques, particularly regarding the accuracy of its cost model, the rate and impact of false-positive exploration, and the memory efficiency of its probabilistic filters? (§5.4)

5.1 Experimental Setup

Basic configuration. We conduct all experiments on a single server with the following specifications:

- **CPU:** 2× 28-core Intel Xeon Gold 6330 @ 2.00GHz;
- **RAM:** 512GB (16× 32GB DDR4 2933MT/s);
- **SSD:** 1× Samsung PM9A3 3.84TB;
- **OS:** Ubuntu 22.04 LTS with Linux kernel 5.15.0.

Datasets. We use three real-world datasets:

1. **YFCC10M** contains image embeddings and labels. It consists of 10 million 192-dimensional `uint8` base vectors and 100,000 query vectors. Each base vector is associated with 1–1517 labels (10.8 on average), while each query has 1–2 labels (1.38 on average). The number of possible labels is 200,386. This dataset is identical to the dataset used in the BigANN benchmark [35].
2. **YT5M** (YouTube-8M)¹ contains video embeddings and labels [1]. We use the training subset, splitting it into 5 million 1024-dimensional `float` base vectors and 10,000 query vectors. Base vectors have 1–23 labels (3.01 on average), and queries have 1–16 labels (3.05 on average). The number of possible labels is 3,862.
3. **LAION100M** contains image embeddings and metadata [31]. We extract the first 100 million vectors from LAION400M as base vectors, and an additional 10,000 vectors as queries. We extract nouns, verbs, adjectives, and adverbs from each image’s text description as labels. Base vectors have 1–405 labels (5.69 on average), and queries have 1–10 labels (5.26 on average). The number of possible labels is 82,558. We also use the image width for range filtering. For queries, we generate $[l, r)$ intervals resulting in selectivities ranging from 0.001% to 50% (median: 15.6%).

Workloads. We construct five types of workloads:

1. **Label** requires the single query label to be present in the returned vectors’ labels. Evaluated on LAION100M, to compare PIPEANN-FILTER against Filtered-DiskANN [15].
2. **LabelAnd** requires all query labels to be a subset of the returned vectors’ labels. Evaluated on YFCC10M, following the BigANN benchmark [35].
3. **LabelOr** requires the query labels to intersect with the returned vectors’ labels (i.e., at least one match). Evaluated on YouTube8M and LAION100M.

¹YouTube-8M shrinks to ~5 million vectors after updates, so we call it YT5M.

4. *Range* requires a specific numerical attribute (image width) of the returned vectors to fall within a query-provided interval $[l, r)$. Evaluated on LAION100M.
5. *Hybrid*: The returned vectors must satisfy either the LabelOr or the Range condition (a union of their results). Evaluated on LAION100M.

Compared systems. We compare PIPEANN-FILTER with representative filtered ANNS systems.

1. *PipeANN-BaseFilter*: An on-SSD graph-based ANNS system built atop PipeANN [17]. We implement its filtering functionality using a heuristic routing strategy: Queries with less than 1% selectivity use accurate pre-filtering, while others fall back to post-filtering. This 1% threshold is chosen based on Figure 2, where pre-filtering and post-filtering show similar performance.
2. *Milvus* [41]: A popular vector database that employs pre-filtering. We deploy Milvus v2.6.11 standalone in a local container. We evaluate both its HNSW (*Milvus-HNSW*) and IVFPQ (*Milvus-IVF*) indexes.
3. *Filtered-DiskANN*: An on-SSD graph-based ANNS system utilizing in-filtering. It stores all the labels in memory and the graph index on the SSD. Because it only supports single-label queries, we restrict its evaluation to the one-label workload on LAION100M.

PIPEANN-FILTER and PipeANN-BaseFilter share the same codebase, using PipeANN’s PipeSearch algorithm for low-latency graph index traversal. They also share the same graph index layout, storing attributes and the index together on the SSD. They construct the index using the unmodified Vamana [37] algorithm. For PIPEANN-FILTER, R_d is selected to make each record with dense neighbors occupy one more 4KB page than the original record. Take the LAION100M for example. Each original record occupies 4056 bytes (~ 4 KB), and each record with dense neighbors occupies 8068 bytes (~ 8 KB). In contrast, Filtered-DiskANN builds a Filtered-Vamana index, which explicitly connects vectors sharing the same labels. We find that using the same R for Filtered-DiskANN and PIPEANN-FILTER leads to poor connectivity and thus harms recall. Therefore, we configure FilteredDiskANN with a larger R (e.g., $R = 256$ for LAION100M). For Milvus-IVF, we set the number of clusters ($nlist$) to $\sim \sqrt{N}$ and PQ bytes to $\text{dims}/2$, following official recommendations [28]. Detailed index build parameters are summarized in Table 2.

Metrics. We evaluate the latency and throughput at different recalls. We search the top-10 nearest neighbors to get the recall (i.e., $\text{recall}_{10@10}$). We mainly compare the systems at 0.9 recall, as recommended by the BigANN [33, 35] benchmark. For latency, we use 1 search thread. For throughput, we use enough search threads to saturate the SSD.

Dataset	Vamana				HNSW		IVFPQ	
	R	R_d	L	B	M	efc	$nlist$	B
YFCC10M	48	1500	72	64	24	200	4096	96
YT5M	48	1800	72	128	24	200	4096	512

Dataset	Vamana				Filtered-Vamana		
	R	R_d	L	B	R	L	B
LAION100M	96	1100	128	64	256	256	64

Table 2. Index build parameters. We omit Milvus-HNSW and Milvus-IVF on LAION100M due to their suboptimal performance on the two smaller datasets. L and efc : candidate pool length during build; B : PQ bytes per vector; R : number of edges per vector (for Vamana); M : number of edges in HNSW layers > 0 ($2M$ edges in layer 0); R_d : number of edges per vector with 2-hop neighbors; $nlist$: number of clusters.

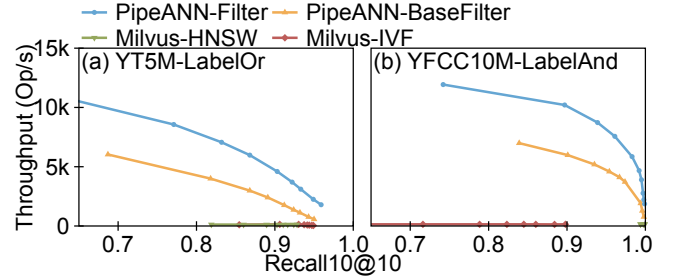


Figure 5. Search throughput on YT5M and YFCC10M.

5.2 Overall Performance

In this section, we evaluate PIPEANN-FILTER on two label-filtering datasets: YT5M and YFCC10M. YT5M evaluates label OR conditions, while YFCC10M evaluates label AND conditions. We compare PIPEANN-FILTER against PipeANN-BaseFilter and Milvus.

Throughput. Figure 5 shows the search throughput. From the figure, we make the following observations:

- (1) PIPEANN-FILTER achieves higher throughput than PipeANN-BaseFilter. At 0.9 recall, PIPEANN-FILTER reaches $1.91\times$ and $1.71\times$ the throughput of PipeANN-BaseFilter on YT5M and YFCC10M, respectively. This improvement comes from PIPEANN-FILTER’s efficient in-filtering powered by speculative filtering. For instance, at 0.9 recall, it processes 77.6% and 50.4% of queries using speculative in-filtering on these two datasets. In contrast, PipeANN-BaseFilter relies entirely on pre- and post-filtering, both of which incur higher I/O overhead than in-filtering for queries with moderate selectivity and recall targets.

At higher recall targets, this throughput advantage grows significantly: PIPEANN-FILTER reaches $3.95\times$ and $2.03\times$ the

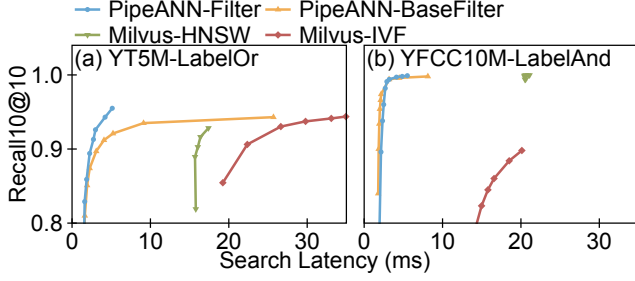


Figure 6. Search latency on YT5M and YFCC10M.

throughput of PipeANN-BaseFilter on YT5M (at 0.95 recall) and YFCC10M (at 0.99 recall). This gap stems from the rapidly increasing cost of post-filtering. While both in-filtering and post-filtering require linearly more I/O to achieve higher accuracy (with a larger L), post-filtering’s cost grows at a much steeper rate (as shown in Table 1).

Furthermore, the performance gain is larger on YT5M than on YFCC10M. This occurs because PipeANN-BaseFilter treats YT5M as a post-filtering-intensive workload (78.8% post-filtering), whereas YFCC10M is pre-filtering-intensive (only 29.7% post-filtering). This highlights the efficiency of speculative in-filtering over post-filtering.

(2) Milvus suffers from low throughput due to its strict pre-filtering policy. Its throughput peaks at under 150 ops/s across all recall targets. While Milvus performs well in standard vector search, its filtered ANNS throughput is severely bottlenecked by heavy attribute scans during pre-filtering. Even with inverted index acceleration, these scans strictly dominate the overall execution time.

Latency. Figure 6 shows the latency results. From the figure, we make the following observations:

(1) PIPEANN-FILTER maintains competitive latency compared to PipeANN-BaseFilter. At 0.9 recall, PIPEANN-FILTER’s average search latency is 72.7% and 1.11 $\times$ of PipeANN-BaseFilter’s latency on YT5M and YFCC10M, respectively. On YT5M, PIPEANN-FILTER reduces latency because it uses in-filtering for moderate-selectivity queries, which creates shorter search paths than PipeANN-BaseFilter’s post-filtering. Conversely, on YFCC10M, PIPEANN-FILTER shifts 20.7% of the total queries from PipeANN-BaseFilter’s pre-filtering to in-filtering. While pre-filtering has poor throughput due to I/O overheads, its latency can be lower under light SSD loads, because its attribute index and vector reads are highly parallelized. In contrast, speculative in-filtering relies on sequential graph traversal.

(2) Milvus exhibits high latency. While it reaches a similar recall to PIPEANN-FILTER, its average latency easily exceeds 10ms, again bottlenecked by attribute index scans. Due to this poor performance, we exclude Milvus from the remaining 100M-scale dataset evaluations.

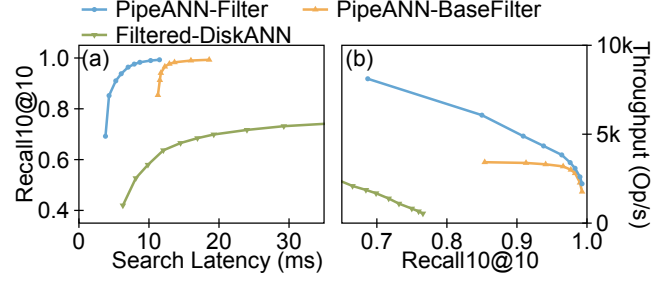


Figure 7. Search latency and throughput of single-label filtering. Dataset: LAION100M.

5.3 100M-Scale Dataset

Single-label performance. Here, we evaluate single-label filtering on LAION100M against PipeANN-BaseFilter and Filtered-DiskANN. Figure 7 shows the results. We find that PIPEANN-FILTER’s performance gains over PipeANN-BaseFilter mirror our previous observations in §5.2: At 0.9 recall, PIPEANN-FILTER achieves 1.44 $\times$ the throughput and reduces average latency to 45.5% of PipeANN-BaseFilter. At the higher 0.99 recall target, these improvements drop slightly to 1.14 $\times$ throughput and 55.1% latency. The throughput trend differs slightly from §5.2 because this workload is heavily pre-filtering-intensive (only 8.49% post-filtering, even less than YFCC10M). While post-filtering I/O still grows faster than PIPEANN-FILTER’s in-filtering, the absolute number of post-filtering queries is so small that the average throughput drop is less severe. Moreover, to achieve high recall, in-filtering requires heavy sequential graph traversal, making it more expensive than pre-filtering (which only needs to read more vectors for re-ranking). As a result, PIPEANN-FILTER selects in-filtering less frequently (e.g., 32.8% at 0.9 recall vs. 18.6% at 0.99 recall) at high recall targets, causing its latency to converge with PipeANN-BaseFilter.

Filtered-DiskANN performs poorly, failing to reach a recall above 0.8. Even though it uses a larger out-degree than other graph indexes, its pruning process still disconnects some vectors sharing the same label. This breaks graph connectivity during in-filtering, severely limiting recall. While PIPEANN-FILTER also relies on speculative in-filtering for many queries (e.g., 32.8% at 0.9 recall), it naturally preserves connectivity using the “bridge” nodes provided by speculative filtering. This allows it to achieve high recall with fewer neighbors.

Other workloads. Figures 8 and 9 show the latency and throughput on other LAION100M workloads. Generally, the performance trends match those of previous evaluations.

The performance gain of PIPEANN-FILTER is most significant in the range-filtering ANNS workload: At 0.9 recall, PIPEANN-FILTER achieves 9.78 $\times$ the throughput and reduces latency to 12.5% of PipeANN-BaseFilter. For PipeANN-BaseFilter, this workload is post-filtering-intensive (75.6%),

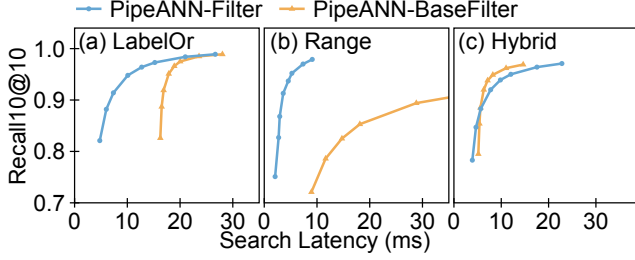


Figure 8. Search latency on LAION100M.

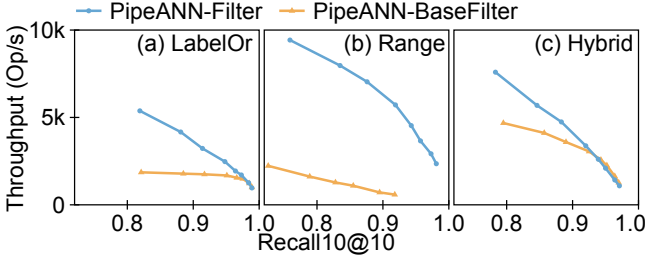


Figure 9. Search throughput on LAION100M.

where PipeANN-BaseFilter fails to reach a high recall within a 10ms latency scale. This shows that post-filtering sometimes struggles to find enough valid nearby vectors under tight range constraints. In contrast, PIPEANN-FILTER’s speculative in-filtering maintains graph connectivity, delivering both superior recall and throughput.

5.4 In-Depth Analysis

In this section, we examine the cost estimation accuracy, the false-positive rate of speculative in-filtering, and the memory usage of PIPEANN-FILTER’s probabilistic filters.

Cost estimation accuracy. We find that the cost estimation for speculative pre-filtering is highly accurate, as its index scan and brute-force search costs are highly predictable. Furthermore, thanks to low query selectivity for all labels, speculative pre-filtering translates into exact attribute index scans in our workloads. Thus, we focus our discussion on in-filtering and post-filtering, specifically comparing their estimated I/O versus actual I/O. We omit the compute cost here because it is proportional to the I/O volume during both graph traversal and attribute index scans.

Figure 10 shows the in-filtering results for two representative workloads, YT5M and LAION100M (other workloads show similar trends). For YT5M, the I/O estimation is quite accurate ($0.74\times$ to $1.04\times$ of actual I/O). However, it is overly conservative for LAION100M (estimating up to $2.05\times$ the actual I/O). This overestimation happens because of PIPEANN-FILTER’s early termination mechanism. The search stops as soon as it finds L valid vectors that cannot be updated

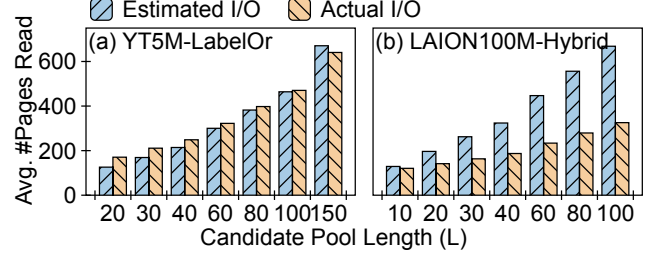


Figure 10. I/O estimation for speculative in-filtering.

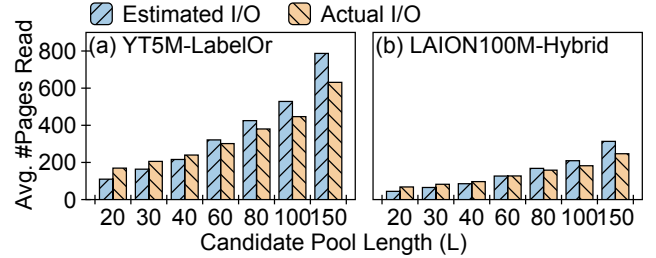


Figure 11. I/O estimation for post-filtering.

by closer neighbors. In contrast, the cost model conservatively assumes a search budget of L/s (where s is selectivity). Although they are statistically equivalent when the valid vectors are uniformly distributed, they are not when the valid vectors show different distributions (e.g., clustered).

Figure 11 shows the results for post-filtering. From the figure, we observe that the estimated I/O initially underestimates the actual I/O, but then grows to overestimate it. The initial underestimation occurs because our model omits the $\log(N)$ baseline cost of traversing the graph toward the target region [40], assuming high search L values will dominate the cost. At larger L values, it overestimates the cost due to the same early termination effect seen in in-filtering.

In conclusion, developing a data-distribution-aware cost model that accounts for early termination is an interesting direction for future work.

False-positive exploration rate. Speculative filtering intentionally explores false-positive vectors. In our setup, this occurs almost entirely during in-filtering, as pre-filtering executes precise index scans. Thus, we focus on the in-filtering false-positive rate. Overall, the false-positive rate ranges from 0.8% to 69.4%, averaging 23.6% with a median of 17.1%. This confirms that PIPEANN-FILTER’s lightweight filters remain highly effective. In the worst-case scenario (LAION100M-1Label), PIPEANN-FILTER experiences an average false-positive rate of 63.5% across high-recall targets (0.85 to 0.99). Despite this high rate, PIPEANN-FILTER’s speculative in-filtering still outperforms the strict pre- and post-filtering of PipeANN-BaseFilter (as shown earlier in Figure 7).

Dataset	Type	In-Memory Filter Size	Ratio to On-SSD Attribute Index
YFCC10M	Label	39MB	3.5%
YT5M	Label	20MB	28.9%
LAION100M	Label	384MB	15.8%
LAION100M	Range	96MB	12.5%

Table 3. Memory usage of PIPEANN-FILTER’s probabilistic filters.

Memory usage. Table 3 describes the memory usage of PIPEANN-FILTER’s probabilistic filters. For label attributes, PIPEANN-FILTER applies per-vector Bloom filters, consuming a fixed 4 bytes per vector. Because the average number of labels per vector varies across datasets (ranging from 3.01 to 10.8), these Bloom filters account for 3.5% to 28.9% of the total attribute index size. Generally, a larger memory footprint correlates with better filtering performance (e.g., YFCC10M vs. YT5M). However, PIPEANN-FILTER remains efficient even on YFCC10M, where the memory-to-SSD index ratio is merely 3.5%. For range attributes, PIPEANN-FILTER quantizes values (from 4 bytes down to 1 byte). This requires only 12.5% of the memory used by the full SSD attribute index (which stores 8 bytes per vector for the ID and raw value).

6 Related Work

On-SSD ANNS. Large-scale vector datasets (e.g., billion-scale [34, 44]) require terabytes of storage space. Storing vectors and indexes on the SSD is a practical approach to support vector search in a scalable and cost-effective manner.

Many existing works focus on graph-based ANNS on the SSD. Typically, they follow a graph layout similar to DiskANN [37]: the full graph is stored on the SSD, while PQ-compressed vectors are kept in memory to guide graph navigation. These works primarily address two hardware characteristics of SSDs compared to DRAM: larger access granularity and higher access latency. First, an SSD’s access unit (a page) is much larger than a single graph record (typically hundreds of bytes), leading to unused space per read. To improve I/O efficiency, systems pack nearby records [43, 47] or additional neighbors [20] into this unused space, thus reducing the number of I/O requests and increasing throughput. Second, an SSD’s high access latency makes the search process heavily I/O-bound. To mitigate this, previous works propose compute-I/O overlapping [7, 17, 43], or aim to reduce the total number of search hops via entry point optimization [19, 43] and early stopping [22]. For a comprehensive survey, we refer readers to [8].

Other works explore cluster-based ANNS on the SSD. These systems group vectors into clusters during index building. During a search, they only scan the clusters whose

centers are closest to the query. Unlike in-memory IVF [2, 3], reading large clusters from the SSD is slow. Therefore, SPANN [6, 46] adopts a fine-grained hierarchical clustering approach, where each cluster contains only tens of vectors (instead of the $\sqrt{N}$ recommended by Faiss). An in-memory graph is then used to index these cluster centers (e.g., a 100M-scale graph for 1 billion vectors). Because cluster-based indexes rely on brute-force scanning within clusters, they can also be compute-intensive. Some recent works accelerate this scanning process using GPUs [39] or SmartSSDs [38].

Unlike the above systems, which focus on standard (unfiltered) top- k vector search, PIPEANN-FILTER addresses the I/O bottlenecks of top- k search *with attribute filters*.

Filtered ANNS. There are two primary approaches to efficient filtered ANNS: attribute-aware vector indexing and attribute-agnostic search.

Attribute-aware vector indexes modify the underlying graph structure. They adjust the edge selection mechanism to connect vectors that are highly likely to be accessed together under specific attribute constraints. These works mainly target two workloads: label filtering and range filtering. For label filtering, systems typically connect vectors that share overlapping labels [5, 15, 42]. Range filtering is more complex. One approach builds the index by explicitly connecting vectors with overlapping ranges [50]. Another approach partitions the dataset into sub-graphs based on value ranges, and then connects these sub-graphs [25, 45]. Because the graph structure is deeply customized for specific attributes, this approach can achieve very high performance. However, building and maintaining separate indexes for different attributes is space-inefficient, and an index optimized for one set of attributes may perform poorly on others.

Attribute-agnostic search relies on a single, general-purpose vector index for all queries, regardless of the attribute constraints. It typically employs the classic mechanisms: pre-filtering, in-filtering, and post-filtering. Due to its high generalizability and low storage overhead, modern vector databases like Faiss [9] and Milvus [41] adopt this approach. Notably, ACORN [29] proposes densifying the standard graph index to maintain connectivity during in-filtering, an insight that inspired PIPEANN-FILTER’s on-SSD graph layout. While highly general, attribute-agnostic search typically shows lower performance compared to attribute-aware approaches [24].

PIPEANN-FILTER follows the attribute-agnostic design for generalizability. By introducing speculative filtering, PIPEANN-FILTER overcomes the I/O bottlenecks that traditionally make pre- and in-filtering impractical on the SSD.

Probabilistic filters. Probabilistic filters, such as Bloom filters [4], are highly compact data structures designed for fast, approximate membership testing. Storage systems, like Log-Structured Merge (LSM) trees [10], adopt them to prevent expensive, unnecessary disk reads. Similarly, distributed systems use them to reduce network I/O [36, 48]. Over time,

many advanced probabilistic filters have emerged. For example, the Cuckoo filter [11] supports item deletion (unlike basic Bloom filters), and SuRF [49] supports both single-key lookups and range queries.

Strict filtering cannot adopt probabilistic filters because it cannot tolerate false positives during the search. In contrast, speculative filtering enables their use based on our key observation that the initial filtering need not be accurate, and that subsequent verification is lightweight.

7 Conclusion

We propose PIPEANN-FILTER, a filtered ANNS system on SSD. Leveraging speculative filtering, PIPEANN-FILTER mitigates the I/O bottleneck of traditional filtering mechanisms. Evaluations show that PIPEANN-FILTER boosts performance compared to existing filtered ANNS systems on SSD. This work shows that design philosophies from traditional storage systems, such as leveraging probabilistic data structures to reduce disk reads, can be repurposed to accelerate modern vector storage systems.

References

- [1] Sami Abu-El-Haija, Nisarg Kothari, Joonseok Lee, Paul Natsev, George Toderici, Balakrishnan Varadarajan, and Sudheendra Vijayanarasimhan. 2016. YouTube-8M: A Large-Scale Video Classification Benchmark. *CoRR* abs/1609.08675 (2016). arXiv:1609.08675 <http://arxiv.org/abs/1609.08675>
- [2] Artem Babenko and Victor Lempitsky. 2012. The inverted multi-index. In *2012 IEEE Conference on Computer Vision and Pattern Recognition*. 3069–3076. doi:10.1109/CVPR.2012.6248038
- [3] Dmitry Baranchuk, Artem Babenko, and Yury Malkov. 2018. Revisiting the inverted indices for billion-scale approximate nearest neighbors. In *Proceedings of the European Conference on Computer Vision (ECCV)*. 202–216.
- [4] Burton H. Bloom. 1970. Space/time trade-offs in hash coding with allowable errors. *Communication of the ACM* 13, 7 (1970), 422–426. doi:10.1145/362686.362692
- [5] Yuzheng Cai, Jiayang Shi, Yizhuo Chen, and Weiguo Zheng. 2024. Navigating Labels and Vectors: A Unified Approach to Filtered Approximate Nearest Neighbor Search. , Article 246 (2024). doi:10.1145/3698822
- [6] Qi Chen, Bing Zhao, Haidong Wang, Mingqin Li, Chuanjie Liu, Zengzhong Li, Mao Yang, and Jingdong Wang. 2021. SPANN: highly-efficient billion-scale approximate nearest neighbor search. In *Proceedings of the 35th International Conference on Neural Information Processing Systems (NIPS '21)*. Curran Associates Inc., Red Hook, NY, USA, Article 398.
- [7] Weijian Chen, Haotian Liu, Yangshen Deng, Long Xiang, Liang Huang, Gezi Li, and Bo Tang. 2026. AlayaLaser: Efficient Index Layout and Search Strategy for Large-scale High-dimensional Vector Similarity Search. arXiv:2602.23342 [cs.DB] <https://arxiv.org/abs/2602.23342>
- [8] Xiaoyu Chen, Jinxiu Qu, Yitong Song, Shuhang Lu, Huiling Li, Minghui Jiang, Wei Zhou, Jianliang Xu, Xuanhe Zhou, and Fan Wu. 2026. Disk-Resident Graph ANN Search: An Experimental Evaluation. arXiv:2603.01779 [cs.DB] <https://arxiv.org/abs/2603.01779>
- [9] Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvassy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. 2024. The Faiss library. (2024). arXiv:2401.08281 [cs.LG]
- [10] Facebook. 2026. RocksDB: A Persistent Key-Value Store for Flash and RAM Storage. <http://rocksdb.org/>.
- [11] Bin Fan, Dave G. Andersen, Michael Kaminsky, and Michael D. Mitzenmacher. 2014. Cuckoo Filter: Practically Better Than Bloom. In *Proceedings of the 10th ACM International Conference on Emerging Networking Experiments and Technologies (CoNEXT '14)*. Association for Computing Machinery, Sydney, Australia, 75–88. doi:10.1145/2674005.2674994
- [12] Wenqi Fan, Yujuan Ding, Liangbo Ning, Shijie Wang, Hengyun Li, Dawei Yin, Tat-Seng Chua, and Qing Li. 2024. A Survey on RAG Meeting LLMs: Towards Retrieval-Augmented Large Language Models. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD '24)*. Association for Computing Machinery, Barcelona, Spain, 6491–6501. doi:10.1145/3637528.3671470
- [13] Cong Fu, Chao Xiang, Changxu Wang, and Deng Cai. 2019. Fast approximate nearest neighbor search with the navigating spreading-out graph. In *Proceedings of the VLDB Endowment (VLDB '19)*. VLDB Endowment, Los Angeles, CA, USA, 461–474. doi:10.14778/3303753.3303754
- [14] Tiezheng Ge, Kaiming He, Qifa Ke, and Jian Sun. 2013. Optimized Product Quantization for Approximate Nearest Neighbor Search. In *2013 IEEE Conference on Computer Vision and Pattern Recognition*. 2946–2953. doi:10.1109/CVPR.2013.379
- [15] Siddharth Gollapudi, Neel Karia, Varun Sivashankar, Ravishankar Krishnaswamy, Nikit Begwani, Swapnil Raz, Yiyong Lin, Yin Zhang, Neelam Mahapatro, Premkumar Srinivasan, Amit Singh, and Harsha Vardhan Simhadri. 2023. Filtered-DiskANN: Graph Algorithms for Approximate Nearest Neighbor Search with Filters. In *Proceedings of the ACM Web Conference 2023 (WWW '23)*. Association for Computing Machinery, Austin, TX, USA, 3406–3416. doi:10.1145/3543507.3583552
- [16] Martin Grohe. 2020. word2vec, node2vec, graph2vec, X2vec: Towards a Theory of Vector Embeddings of Structured Data. In *Proceedings of the 39th ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems (, Portland, OR, USA,) (PODS '20)*. Association for Computing Machinery, New York, NY, USA, 1–16. doi:10.1145/3375395.3387641
- [17] Hao Guo and Youyou Lu. 2025. Achieving Low-Latency Graph-Based Vector Search via Aligning Best-First Search Algorithm with SSD. In *19th USENIX Symposium on Operating Systems Design and Implementation (OSDI '25)*. USENIX Association, Boston, MA, USA.
- [18] Yupeng Hou, Jiacheng Li, Zhankui He, An Yan, Xiuxi Chen, and Julian J. McAuley. 2024. Bridging Language and Items for Retrieval and Recommendation. *CoRR* abs/2403.03952 (2024). arXiv:2403.03952 doi:10.48550/ARXIV.2403.03952
- [19] Haodi Jiang, Hao Guo, Minhui Xie, Jiwei Shu, and Youyou Lu. 2026. High-Throughput, Cost-Effective Billion-Scale Vector Search with a Single GPU. In *Proceedings of the 2026 International Conference on Management of Data (SIGMOD '26)*. Association for Computing Machinery, Bengaluru, India.
- [20] Dingyi Kang, Dongming Jiang, Hanshen Yang, Hang Liu, and Bingzhe Li. 2025. Scalable Disk-Based Approximate Nearest Neighbor Search with Page-Aligned Graph. arXiv:2509.25487 [cs.LG] <https://arxiv.org/abs/2509.25487>
- [21] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Proceedings of the 34th International Conference on Neural Information Processing Systems (NIPS '20)*. Curran Associates Inc., Vancouver, BC, Canada, Article 793.
- [22] Conglong Li, Minjia Zhang, David G. Andersen, and Yuxiong He. 2020. Improving Approximate Nearest Neighbor Search through Learned Adaptive Early Termination. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data (SIGMOD '20)*. Association for Computing Machinery, Portland, OR, USA, 2539–2554. doi:10.1145/3318464.3380600

- [23] Jie Li, Haifeng Liu, Chuanghua Gui, Jianyu Chen, Zhenyuan Ni, Ning Wang, and Yuan Chen. 2018. The Design and Implementation of a Real Time Visual Search System on JD E-commerce Platform. In *Proceedings of the 19th International Middleware Conference Industry (Middleware '18)*. Association for Computing Machinery, Rennes, France, 9–16. doi:10.1145/3284028.3284030
- [24] Mocheng Li, Xiao Yan, Baotong Lu, Yue Zhang, James Cheng, and Chenhao Ma. 2026. Attribute Filtering in Approximate Nearest Neighbor Search: An In-depth Experimental Study. In *Proceedings of the 2026 International Conference on Management of Data (SIGMOD '26)*. Association for Computing Machinery, Bengaluru, India.
- [25] Anqi Liang, Pengcheng Zhang, Bin Yao, Zhongpu Chen, Yitong Song, and Guangxu Cheng. 2024. UNIFY: Unified Index for Range Filtered Approximate Nearest Neighbors Search. *Proceedings of the VLDB Endowment* (2024), 1118–1130. doi:10.14778/3717755.3717770
- [26] Yu A. Malkov and D. A. Yashunin. 2020. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI)* 42, 4 (2020), 824–836. doi:10.1109/TPAMI.2018.2889473
- [27] Tomás Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. 2013. Efficient Estimation of Word Representations in Vector Space. In *1st International Conference on Learning Representations, Workshop Track Proceedings (ICLR '13)*. Scottsdale, Arizona, USA. <http://arxiv.org/abs/1301.3781>
- [28] Milvus. 2026. IVF_PQ. <https://milvus.io/docs/ivf-pq.md>.
- [29] Liana Patel, Peter Kraft, Carlos Guestrin, and Matei Zaharia. 2024. ACORN: Performant and Predicate-Agnostic Search Over Vector Embeddings and Structured Data. , Article 120 (2024). doi:10.1145/3654923
- [30] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. 2021. Learning Transferable Visual Models From Natural Language Supervision. In *Proceedings of the 38th International Conference on Machine Learning (ICML '21)*. PMLR, Virtual Event, 8748–8763. <https://proceedings.mlr.press/v139/radford21a.html>
- [31] Christoph Schuhmann, Richard Vencu, Romain Beaumont, Robert Kaczmarczyk, Clayton Mullis, Aarush Katta, Theo Coombes, Jenia Jitsev, and Aran Komatsuzaki. 2021. LAION-400M: Open Dataset of CLIP-Filtered 400 Million Image-Text Pairs. *CoRR* abs/2111.02114 (2021). arXiv:2111.02114 <https://arxiv.org/abs/2111.02114>
- [32] P. Griffiths Selinger, M. M. Astrahan, D. D. Chamberlin, R. A. Lorie, and T. G. Price. 1979. Access path selection in a relational database management system. In *Proceedings of the 1979 ACM SIGMOD International Conference on Management of Data (SIGMOD '79)*. Association for Computing Machinery, Boston, Massachusetts, 23–34. doi:10.1145/582095.582099
- [33] Harsha Simhadri. 2021. Results of the NeurIPS'21 Challenge on Billion-Scale Approximate Nearest Neighbor Search. In *Proceedings of the 35th International Conference on Neural Information Processing Systems (NIPS '21)*. Curran Associates Inc., Red Hook, NY, USA.
- [34] Harsha Simhadri. 2022. Research talk: Approximate nearest neighbor search systems at scale. <https://www.youtube.com/watch?v=BnYNdSIKibQ&list=PLD7HFcN7LXREJTWFKYqwMcCc1nZKIXBo9&index=9>.
- [35] Harsha Vardhan simhadri, Martin Aumüller, Matthijs Douze, Dmitry Baranchuk, Amir Ingber, Edo Liberty, George Williams, Ben Landrum, Magdalen Dobson Manohar, Mazin Karjkar, Laxman Dhulipala, Meng Chen, Yue Chen, Rui Ma, Kai Zhang, Yuzheng Cai, Jiayang Shi, Weiguo Zheng, Yizhuo Chen, Jie Yin, and Ben Huang. 2025. Results of the Big ANN: NeurIPS'23 competition. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems Datasets and Benchmarks Track*. <https://openreview.net/forum?id=dB6W56wQL9>
- [36] Haoyu Song, Sarang Dharmapurikar, Jonathan Turner, and John Lockwood. 2005. Fast hash table lookup using extended bloom filter: an aid to network processing. In *Proceedings of the 2005 Conference on Applications, Technologies, Architectures, and Protocols for Computer Communications (SIGCOMM '05)*. Association for Computing Machinery, Philadelphia, Pennsylvania, USA, 181–192. doi:10.1145/1080091.1080114
- [37] Suhas Jayaram Subramanya, Devvrit, Rohan Kadekodi, Ravishankar Krishaswamy, and Harsha Vardhan Simhadri. 2019. DiskANN: fast accurate billion-point nearest neighbor search on a single node. In *Proceedings of the 33rd International Conference on Neural Information Processing Systems (NIPS '19)*. Curran Associates Inc., Red Hook, NY, USA, Article 1233.
- [38] Bing Tian, Haikun Liu, Zhuohui Duan, Xiaofei Liao, Hai Jin, and Yu Zhang. 2024. Scalable Billion-point Approximate Nearest Neighbor Search Using SmartSSDs. In *2024 USENIX Annual Technical Conference (USENIX ATC '24)*. USENIX Association, Santa Clara, CA, 1135–1150. <https://www.usenix.org/conference/atc24/presentation/tian>
- [39] Bing Tian, Haikun Liu, Yuhang Tang, Shihai Xiao, Zhuohui Duan, Xiaofei Liao, Hai Jin, Xuechang Zhang, Junhua Zhu, and Yu Zhang. 2025. Towards High-throughput and Low-latency Billion-scale Vector Search via CPU/GPU Collaborative Filtering and Re-ranking. In *23rd USENIX Conference on File and Storage Technologies (FAST '25)*. USENIX Association, Santa Clara, CA, 171–185. <https://www.usenix.org/conference/fast25/presentation/tian-bing>
- [40] Godfried T. Toussaint. 1980. The relative neighbourhood graph of a finite planar set. *Pattern Recognition* 12, 4 (1980), 261–268. doi:10.1016/0031-3203(80)90066-7
- [41] Jianguo Wang, Xiaomeng Yi, Rentong Guo, Hai Jin, Peng Xu, Shengjun Li, Xiangyu Wang, Xiangzhou Guo, Chengming Li, Xiaohai Xu, Kun Yu, Yuxing Yuan, Yinghao Zou, Jiquan Long, Yudong Cai, Zhenxiang Li, Zhifeng Zhang, Yihua Mo, Jun Gu, Ruiyi Jiang, Yi Wei, and Charles Xie. 2021. Milvus: A Purpose-Built Vector Data Management System. In *Proceedings of the 2021 International Conference on Management of Data (SIGMOD '21)*. Association for Computing Machinery, Virtual Event, 2614–2627. doi:10.1145/3448016.3457550
- [42] Mengzhao Wang, Lingwei Lv, Xiaoliang Xu, Yuxiang Wang, Qiang Yue, and Jiongfeng Ni. 2023. An Efficient and Robust Framework for Approximate Nearest Neighbor Search with Attribute Constraint. In *Advances in Neural Information Processing Systems*, A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine (Eds.), Vol. 36. Curran Associates, Inc., 15738–15751. https://proceedings.neurips.cc/paper_files/paper/2023/file/32e41d6b0a51a63a9a90697da19d235d-Paper-Conference.pdf
- [43] Mengzhao Wang, Weizhi Xu, Xiaomeng Yi, Songlin Wu, Zhangyang Peng, Xiangyu Ke, Yunjun Gao, Xiaoliang Xu, Rentong Guo, and Charles Xie. 2024. Starling: An I/O-Efficient Disk-Resident Graph Index Framework for High-Dimensional Vector Similarity Search on Data Segment. In *Proceedings of the ACM on Management of Data (SIGMOD '24)*. Association for Computing Machinery, Santiago, Chile. doi:10.1145/3639269
- [44] Chuangxian Wei, Bin Wu, Sheng Wang, Renjie Lou, Chaoqun Zhan, Feifei Li, and Yuanzhe Cai. 2020. AnalyticDB-V: a hybrid analytical engine towards query fusion for structured and unstructured data. In *Proceedings of the VLDB Endowment (VLDB '20)*. VLDB Endowment, Tokyo, Japan, 3152–3165. doi:10.14778/3415478.3415541
- [45] Yuexuan Xu, Jianyang Gao, Yutong Gou, Cheng Long, and Christian S. Jensen. 2024. iRangeGraph: Improvising Range-dedicated Graphs for Range-filtering Nearest Neighbor Search. In *Proceedings of the ACM on Management of Data (SIGMOD '24)*. Association for Computing Machinery, Santiago, Chile. doi:10.1145/3698814
- [46] Yuming Xu, Hengyu Liang, Jin Li, Shuotao Xu, Qi Chen, Qianxi Zhang, Cheng Li, Ziyue Yang, Fan Yang, Yuqing Yang, Peng Cheng, and Mao Yang. 2023. SPFresh: Incremental In-Place Update for Billion-Scale Vector Search. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP '23)*. Association for Computing Machinery,

- Koblenz, Germany, 545–561. doi:10.1145/3600006.3613166
- [47] Peiqi Yin, Xiao Yan, Qihui Zhou, Hui Li, Xiaolu Li, Lin Zhang, Meiling Wang, Xin Yao, and James Cheng. 2025. Gorgeous: Revisiting the Data Layout for Disk-Resident High-Dimensional Vector Search. *arXiv preprint arXiv:2508.15290* (2025).
- [48] Minlan Yu, Alex Fabrikant, and Jennifer Rexford. 2009. BUFFALO: bloom filter forwarding architecture for large organizations. In *Proceedings of the 5th International Conference on Emerging Networking Experiments and Technologies (CoNEXT '09)*. Association for Computing Machinery, New York, NY, USA, 313–324. doi:10.1145/1658939.1658975
- [49] Huanchen Zhang, Hyeontaek Lim, Viktor Leis, David G. Andersen, Michael Kaminsky, Kimberly Keeton, and Andrew Pavlo. 2018. SuRF: Practical Range Query Filtering with Fast Succinct Tries. In *Proceedings of the 2018 International Conference on Management of Data (SIGMOD '18)*. Association for Computing Machinery, Houston, TX, USA, 323–336. doi:10.1145/3183713.3196931
- [50] Chaoji Zuo, Miao Qiao, Wenchao Zhou, Feifei Li, and Dong Deng. 2024. SeRF: Segment Graph for Range-Filtering Approximate Nearest Neighbor Search. In *Proceedings of the ACM on Management of Data (SIGMOD '24)*. Association for Computing Machinery, Santiago, Chile, Article 69. doi:10.1145/3639324